

Юнит-тестирование C++ и Python

Фреймворк юнит-тестирования призван учесть все накопившиеся за время эксплуатации проблемы и потребности и реализовать новую, более гибкую систему юнит-тестирования, решающую следующие задачи:

1. Автоматизация развертывания и заполнение баз данных.
2. Тестирование запуска сервисов и межсервисного взаимодействия.
3. Систематизация структуры тестов (исходные коды, ресурсы, рабочий каталог, прочее).
4. Сборка тестов в виде отдельных исполняемых файлов на заранее собранной платформе.

Исходный код тестового фреймворка лежит в репозитории Wasaby Framework — [git@git.sbis.ru:sbis/core.git](https://git.sbis.ru/sbis/core.git) в каталоге `tools/test_framework`.

Для прикладной разработки фреймворк юнит-тестирования поставляется вместе с [SBIS SDK](#).



Для лучшего понимания основ работы с юнит-тестами рекомендуем ознакомиться с [вебинаром по внедрению платформенных юнит-тестов для прикладной БЛ](#).

Настройка рабочего места

Перед началом работы с юнит-тестами выполните [настройку рабочего места](#) по инструкции, в зависимости от операционной системы.

Описание

Фреймворк представляет собой Python-пакет, который может быть запущен в качестве отдельного приложения или встроен в другие скрипты на Python.

В состав пакета входят:

- асинхронный [веб-сервер Tornado](#), с помощью которого реализована поддержка [mock-сервисов](#);
- утилита работы с БД `sbis-psql` с максимально близким к `psql` интерфейсом;
- набор платформоспецифичных сторонних библиотек ядра СБИС (`boost`, `rabbitmq`).

Для выполнения тестов, требующих развертывания БД и/или сайтов, фреймворку необходим локальный дистрибутив `Jinnee` , путь к которому может быть передан через параметры командной строки или переменные окружения.

Для создания C++ тестов вместе с `boost::unit_tests` также можно использовать две библиотеки для написания моков классов. Они поставляются вместе с пакетом `boost`.

1. [turtle](#) - для использования подключите `#include <turtle/mock.hpp>`.
2. [fakeit](#) - для использования подключите `#include <fakeit/fakeit.hpp>`.

В общем случае для выполнения тестов пользователю достаточно через параметры командной строки или параметры вызова функции указать только каталог с тестовыми сценариями. Далее тестовый фреймворк все сделает сам: развернет и заполнит БД, развернет и запустит сайты, настроит и запустит mock-сервисы и сервер Redis.

Пользователю и разработчикам тестов не нужно заботиться об именах БД и номерах портов сайтов/сервисов, т. к. фреймворк развернет БД с генерируемыми именами и запустит сайты/сервисы на случайных/свободных портах.

Правила организации тестового проекта

В целях унификации структуры репозитория, тестовые проекты принято размещать в подкаталоге **tests** в корне репозитория.

Базовый набор данных

Проиллюстрируем пример организации файлов на абстрактном проекте `my_project`.

1. В корне репозитория должна лежать папка **tests**, а внутри неё — папки, названные по именам тестовых проектов, одна папка — это один тестовый проект:

```
├── <root> // корень репозитория
│   └── tests
│       ├── my_test_suite
│       └── my_super_test_suite
```

my_test_suite и **my_super_test_suite** — это отдельные тестовые проекты.

2. В корне каждого тестового проекта должны лежать файл **test.entry** и каталог с исходниками тестов **src**:

```
├── <root> // корень репозитория
│   └── tests
│       ├── CMakeLists.txt
│       └── my_test_suite
│           ├── src
│           │   └── CMakeLists.txt
│           └── test.entry
```

Файл **test.entry** — это текстовый файл в кодировке UTF-8, содержащий описание теста в виде JSON-объекта. В нем могут быть заданы параметры для разворота БД, мок-сервисов, сайтов, сервера Redis и другие.

Прерывание теста по истечении таймаута задается в файле с помощью параметра `timeout` (развертывание БД и сервисов не учитывается).

Подробнее о содержании файла **test.entry** можно прочитать в примерах тестов на [C++](#) и [Python](#).

В простейшем случае файл **test.entry** для C++ содержит пустой объект `{}`, а для Python имя тестового облака (`"test_cloud"`):

```
1  {
2      "pytest_on": "test_cloud"
3  }
```

В каталоге **src** должны содержаться:

- исходные коды и файл сборки тестов **CMakeLists.txt**, если тесты реализованы на C++;
- Python-модули, если тесты реализованы на Python.

3. Правилom хорошего тона считается создание файла **CMakeLists.txt** в каталоге **tests**, внутри которого подключены все тестовые проекты. В примере выше его содержимое будет выглядеть так:

```
add_subdirectory(my_test_suite/src)
```

Дополнительные файлы проекта

Также в каталоге тестового проекта могут содержаться следующие файлы:

1. В каталоге **clouds** должны располагаться все метаданные проектов, используемых в тестах. Например, для тестов на Python:

```
├── <root> // корень репозитория
│   └── tests
│       ├── my_test_suite
│       │   ├── src
│       │   │   ├── CMakeLists.txt
│       │   │   └── simple_test.py
│       │   ├── test.entry
│       │   └── clouds
│       │       └── test_cloud
│       │           ├── test_cloud.s3cld
│       │           ├── test.s3srv
│       │           └── sbis-rpc-service.ini
```



Важно!

Название папки, в которой хранится тестовый проект, должно совпадать с названием файла `*.s3cld`.

Внутри каждого каталога с метаданными сервиса может располагаться файл **sbis-rpc-service.ini** с параметрами тестового облака. Например, чтобы включить отладочное логирование, запишите в файле:

- 1 **[Ядро.Логирование]**
- 2 **Уровень=Отладочный**

2. В папке **setup** лежит скрипт **__main__.py**, который запускается сразу после разворота всей структуры каталогов теста, но до момента запуска тестов и сервисов.

```
|— <root> // корень репозитория
|   |— tests
|       |— MyTestSuite
|           |— src
|               |— CMakeLists.txt
|               |— simple_test.py
|           |— test.entry
|           |— setup
|               |— __main__.py
```

Скрипт используется для решения одной из следующих (или похожих) задач:

- Копирование ресурса из стороннего каталога/репозитория в каталог теста.
- Создание нового файла в каталоге теста.
- Запуск сторонних процессов, которые необходимы для теста. **Крайне не рекомендуется к использованию.**

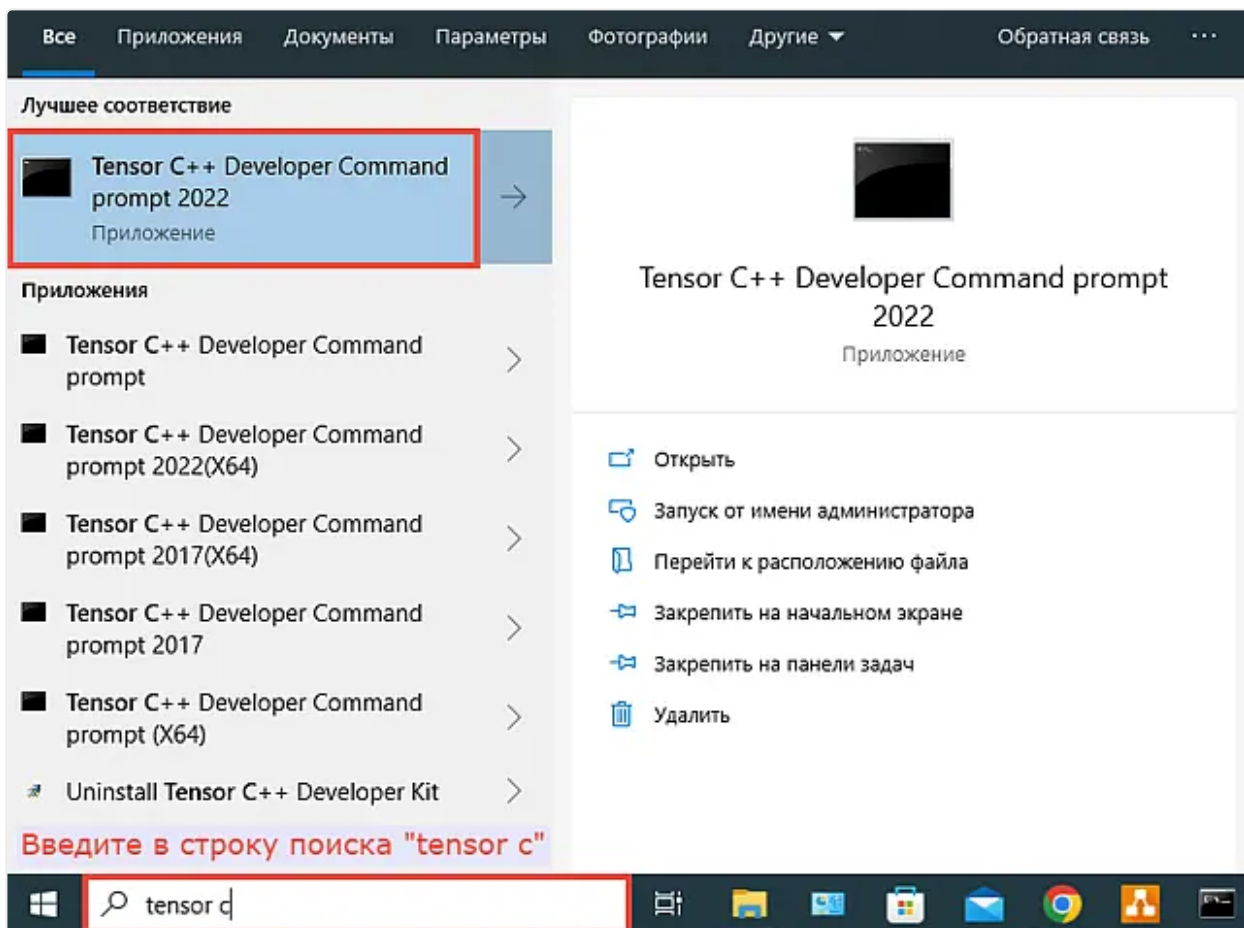
3. В каталог **resources** помещаются дополнительные файлы, на которые после запуска теста будут созданы символические ссылки. Также туда помещается файл **sbis-rpc-service.ini** для того, чтобы добавить параметры к исполняемому файлу тестов, например, параметры логирования:

```
|— <root> // корень репозитория
|   |— tests
|       |— my_test_suite
|           |— src
|               |— CMakeLists.txt
|               |— simple_test.py
|           |— test.entry
|           |— resources
|               |— sbis-rpc-service.ini
|               |— hello_world.txt
```

Запуск тестов

Перед запуском фреймворка

- Для Windows:
откройте консоль [Tensor C++ Developer Command prompt 2025](#):



- Для Linux:

откройте консоль и [добавьте переменные окружения](#):

```
1 export CC=/opt/clang-15.0.6/bin/clang
2 export CXX=/opt/clang-15.0.6/bin/clang++
3 export PATH=${PATH}:/opt/python3.11/bin/:/opt/clang-15.0.6/bin/:/opt/lld-15.0.6/bin/:/opt/llvm-15.0.6/bin/
```

Команды запуска тестов

▼ Все параметры запуска тестов

Таблица 1. Опциональные параметры.

Имя	Описание
tests	Путь к каталогу, в котором расположены тестовые проекты.
test	Имя тестового проекта. Входит в каталог тестовых проектов.
mode	Режим запуска теста. Значения: auto (по умолчанию) и debug (режим отладки). Включить режим отладки, при котором для каждого теста будет создано только тестовое окружение без запуска самого теста. Имеет смысл только при интерактивной работе с консолью.
-t	Полное имя тестового сценария (теста) из тестового проекта. Может быть указан несколько раз, что соответствует запуску тестера с аргументом --run_test=test1,...,testN Подробнее про опции запуска конкретного теста читайте здесь .
-c	Параметр указывает, что необходимо выполнить поиск и удаление "мусорных" БД, т. е. БД оставшихся в

	результате аварийных прерываний работы тестовой оболочки. Поиск производится параллельно с выполнением тестов. Данный параметр рекомендуется всегда устанавливать при выполнении тестов на серверах сборки.
-j, --jobs <число>	Количество параллельно выполняемых тестов (количество потоков). По умолчанию тесты выполняются последовательно в одном потоке.
--jinnee <строка>	Путь к каталогу, в котором находится jinnee-utility . Если параметр определен несколько раз, то поиск jinnee-utility будет произведен по всем указанным каталогам. По умолчанию поиск jinnee-utility производится в каталоге, указанном в переменной окружения Jinnee .
--verbose	Выводить подробную информацию о ходе выполнения тестов. Для упрощения выявления и решения проблем данный параметр рекомендуется всегда устанавливать при выполнении тестов на серверах сборки.
--gcov_flush	Тестирования предполагает сброс информации о покрытии кода с сервисов. Используется для анализа покрытия кода, когда ядро собрано с параметром - DSBIS_COVERAGE_ANALYSIS=1. Используется только в ядре.
--amqp <строка>	Адрес AMQP-брокера для обмена сообщениями. По умолчанию установлен адрес "cloud_service:cloud_service@test-rabbitmq.unix.tensor.ru" .
--xunit_output <строка>	Генерировать результаты выполнения тестов в формате xUnit для интеграции с системами сборки. Используется в сборках на Jenkins.
--attempts <число>	Количество повторных попыток запусков теста в случае получения ошибки в процессе тестирования. По умолчанию 1.
--architecture <число>	Архитектура процессора, нужна в случае кросс-компиляции например macos x86_64 на arm64.
--source_replacement <число>	Принимает два значения from и to заменяет пути до исходников.
--build_dir_replacement <число>	Принимает два значения from и to заменяет пути до сборочной директории.
--skip <число>	Пропустить указанные тесты.
--py_coverage <число>	Включить измерение покрытия кода для Python фреймворком coverage.py. По умолчанию False.
--py_coverage_format <число>	Формат вывода отчета о покрытии кода на Python фреймворком coverage.py(html, xml, json, ...). По умолчанию html.
--timeout <число>	Консольный параметр, влияющий на запуск всего фреймворка, прерывает все запущенные процессы по истечении таймута.

По умолчанию для подключения к серверу БД используются следующие параметры:

- хост "localhost",
- порт "5432",
- пользователь "postgres",
- пароль "<пароль вашей локальной БД>".

Эти параметры могут быть изменены переменными окружения PgHost, PgPort, PgUser, PgPassword соответственно.

Процесс запуска тестов

Запуск теста состоит из трех этапов:

1. Запуск [CMake](#) - для генерации проектных файлов:

- Windows 32bit:

```
1 cmake ..\my_project\tests\<имя тестового проекта>\src ^
2 -DSBIS_UNIT_TESTS=1 -G Ninja ^
3 -DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-
  i686.cmake"
```

- Windows 64bit:

```
1 cmake ..\my_project\tests\<имя тестового проекта>\src ^
2 -DSBIS_UNIT_TESTS=1 -G Ninja ^
3 -DSBIS_ARCHITECTURE=x86_64
```

- Linux и macOS на Intel (x86_64):

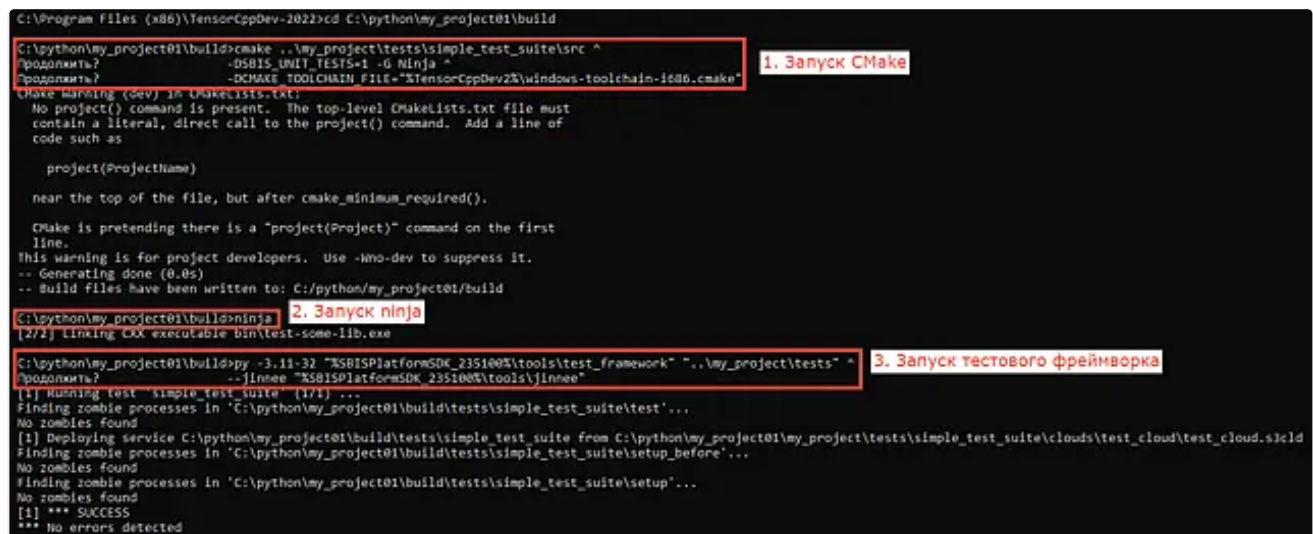
```
1 cmake ../my_project/tests/<имя тестового проекта>/src \
2 -DSBIS_UNIT_TESTS=1 -G Ninja
```

- macOS на Apple M1 (ARM):

```
1 cmake ../my_project/tests/<имя тестового проекта>/src \
2 -DSBIS_ARCHITECTURE=arm64 \
3 -DSBIS_UNIT_TESTS=1 -G Ninja
```

2. ninja - для сборки тестового проекта;

3. Тестовый фреймворк - для запуска тестов.



```
C:\Program Files (x86)\TensorCppDev-2022>cd C:\python\my_project01\build
C:\python\my_project01\build>cmake ..\my_project\tests\simple_test_suite\src ^
  -DSBIS_UNIT_TESTS=1 -G Ninja ^
  -DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-i686.cmake"
CMake Warning (dev) in CMakeLists.txt:
  No project() command is present.  The top-level CMakeLists.txt file must
  contain a literal, direct call to the project() command.  Add a line of
  code such as

    project(ProjectName)

  near the top of the file, but after cmake_minimum_required().

  CMake is pretending there is a "project(Project)" command on the first
  line.
This warning is for project developers.  Use -Wno-dev to suppress it.
-- Generating done (0.0s)
-- Build files have been written to: C:\python\my_project01\build

C:\python\my_project01\build>ninja
[2/2] Linking CXX executable bin\test-some-lib.exe

C:\python\my_project01\build>python -3.11-32 "C:\Program Files (x86)\TensorCppDev-2022\tools\test_framework" "..\my_project\tests" ^
  --jlnnee "C:\Program Files (x86)\TensorCppDev-2022\tools\jlnnee"
[1] Running test simple_test_suite (1/1) ...
Finding zombie processes in 'C:\python\my_project01\build\tests\simple_test_suite\test'...
No zombies found
[1] Deploying service C:\python\my_project01\build\tests\simple_test_suite from C:\python\my_project01\my_project\tests\simple_test_suite\clouds\test_cloud\test_cloud.s3cid
Finding zombie processes in 'C:\python\my_project01\build\tests\simple_test_suite\setup_before'...
No zombies found
Finding zombie processes in 'C:\python\my_project01\build\tests\simple_test_suite\setup'...
No zombies found
[1] *** SUCCESS
*** No errors detected
```

Каждый тест выполняется в собственном окружении, которое создается во временном каталоге **run**. Перед каждым запуском тестового сценария содержимое каталога **run** удаляется. Окружение формируется следующим образом:

1. Если есть файл сборки **src/CMakeLists.txt**, то по нему рекурсивно определяются все зависимые библиотеки, на которые в каталоге **run** создаются символические ссылки.
2. Создаются символические ссылки на все ресурсы из каталога **resources** (кроме файла **sbis-rpc-service.ini**, который копируется).
3. В зависимости от содержимого файла **test.entry** может быть:
 - настроен и запущен mock-сервер;
 - запущен сервер Redis;
 - развернута БД по облакам из каталога **clouds**;
 - развернут в каталоге **run** и запущены сайт по облакам из каталога **clouds**.

Если ваш тест зависит только от SDK и других Python-модулей, то генерация и сборка файлов проекта не требуются. В этом случае запуск теста выполняется из директории, в которой размещен тест.

**Важно!**

Консоль Tensor C++ Developer Command prompt 2025 необходимо запускать от имени администратора.

Запуск всех тестов продукта

- Windows

```
1 py -3.11-32 "<путь к директории SDK>\tools\test_framework" ^
2 "<путь к каталогу с тестовыми проектами>" ^
3 --jinnee "<путь к директории SDK>\tools\jinnee"
```

- Linux

```
1 /opt/python3.11/bin/python3.11 \
2 "<путь к директории SDK>/tools/test_framework" \
3 "<путь к каталогу с тестовыми проектами>" \
4 --jinnee "<путь к директории SDK>/tools/jinnee"
```

- macOS

```
1 python3.11 "<путь к директории SDK>/tools/test_framework" \
2 "<путь к каталогу с тестовыми проектами>" \
3 --jinnee "<путь к директории SDK>/tools/jinnee" \
```

Запуск одного тестового проекта

Один тестовый проект — это имя каталога, где лежит файл **test.entry**. В данном примере, тестовый проект — **MyTestSuite**.

```
|— <root> // корень репозитория
|   |— tests
|       |— my_test_suite
|           |— src
|               |— CMakeLists.txt
|               |— simple_test.py
|               |— another_test.py
|               |— test.entry
```

- Windows

```
1 py -3.11-32 "<путь к директории SDK>\tools\test_framework" ^
2 "<путь к каталогу с тестовыми проектами>" ^
3 "<имя тестового проекта>" ^
4 --jinnee "<путь к директории SDK>\tools\jinnee"
```

- Linux

```
1 /opt/python3.11/bin/python3.11 "<путь к директории
SDK>/tools/test_framework" \
2 "<путь к каталогу с тестовыми проектами>" \
3 "<имя тестового проекта>" \
4 --jinnee "<путь к директории SDK>/tools/jinnee"
```

- macOS

```
1 python3.11 "<путь к директории SDK>/tools/test_framework"
2 "<путь к каталогу с тестовыми проектами>"
3 "<имя тестового проекта>"
4 --jinnee "<путь к директории SDK>/tools/jinnee"
```

Для Python отдельный тестовый сценарий запустить нельзя, можно запускать тесты помодульно (один тестовый модуль). Для этого в параметр **-t** передайте имя тестового модуля, т.е. имя файла, где лежат тесты.

- Windows

```
1 py -3.11-32 "<путь к директории SDK>\tools\test_framework" ^
```

```
2 "<путь к каталогу с тестовыми проектами>" ^
3 "<имя тестового проекта>" ^
4 --jinnee "<путь к директории SDK>\tools\jinnee" ^
5 -t "<имя файла>"
```

- Linux

```
1 /opt/python3.11/bin/python3.11 "<путь к директории
SDK>/tools/test_framework" \
2 "<путь к каталогу с тестовыми проектами>" \
3 "<имя тестового проекта>" \
4 --jinnee "<путь к директории SDK>/tools/jinnee" \
5 -t "<имя файла>"
```

- macOS

```
1 python3.11 "<путь к директории SDK>/tools/test_framework" \
2 "<путь к каталогу с тестовыми проектами>" \
3 "<имя тестового проекта>" \
4 --jinnee "<путь к директории SDK>/tools/jinnee" \
5 -t "<имя файла>"
```

Запуск теста в режиме отладки

Для запуска теста в режиме отладки необходимо передать параметр **debug** тестовому фреймворку.

- Windows

```
1 py -3.11-32 "<путь к директории SDK>\tools\test_framework" ^
2 "<путь к каталогу с тестовыми проектами>" ^
3 "<имя тестового проекта>" debug ^
4 --jinnee "<путь к директории SDK>\tools\jinnee"
```

- Linux

```
1 /opt/python3.11/bin/python3.11 "<путь к директории
SDK>/tools/test_framework" \
2 "<путь к каталогу с тестовыми проектами>" \
3 "<имя тестового проекта>" debug \
4 --jinnee "<путь к директории SDK>/tools/jinnee/"
```

- macOS

```
1 python3.11 "<путь к директории SDK>/tools/test_framework" \
2 "<путь к каталогу с тестовыми проектами>" \
3 "<имя тестового проекта>" debug \
4 --jinnee "<путь к директории SDK>/tools/jinnee"
```

В данном режиме тестовое окружение (БД, сайты, mock-сервисы, Redis) будет существовать до тех пор, пока пользователь не нажмет любую клавишу в консоли. Режим отладки полезен для поиска и устранения ошибок, т. к. позволяет вручную многократно запускать тест и следить за ходом его выполнения.

Пример запуска теста в режиме отладки:


```

C:\Program Files (x86)\TensorCppDev-2022>cd C:\python\my_project01\build
C:\python\my_project01\build>cmake ..\my_project\tests\simple_test_suite\src ^
Продолжить? -DSBIS_UNIT_TESTS=1 -G Ninja ^
Продолжить? -DCMAKE_TOOLCHAIN_FILE="X:\TensorCppDev2\windows-toolchain-1686.cmake"
CMake Warning (dev) in CMakeLists.txt:
-- Configuring done (27.5s)
-- Generating done (0.0s)
-- Build files have been written to: C:\python\my_project01\build

C:\python\my_project01\build>ninja
[2/2] Linking CXX executable bin\test-some-lib.exe

C:\python\my_project01\build>py -3.11-32 "X:\SBISPlatformSDK_235100\tools\test_framework" "..\my_project\tests" ^
Продолжить? simple_test_suite Debug ^
Продолжить? --jinnee "X:\SBISPlatformSDK_235100\tools\jinnee"
[1] Running test 'simple_test_suite' (1/1) ...
Finding zombie processes in "C:\python\my_project01\build\tests\simple_test_suite\test"...
No zombies found
[1] Deploying service C:\python\my_project01\build\tests\simple_test_suite from C:\python\my_project01\my_project\tests\simple_test_suite\clouds\test_cloud\test_cloud.s3cld
Finding zombie processes in "C:\python\my_project01\build\tests\simple_test_suite\setup"...
No zombies found
[1] *** SUCCESS
*** No errors detected

C:\python\my_project01\build>py -3.11-32 "X:\SBISPlatformSDK_235100\tools\test_framework" "..\my_project\tests" ^
Продолжить? simple_test_suite Debug ^
Продолжить? --jinnee "X:\SBISPlatformSDK_235100\tools\jinnee"
[1] Running test 'simple_test_suite' (1/1) ...
Finding zombie processes in "C:\python\my_project01\build\tests\simple_test_suite\test"...
No zombies found
[1] Deploying service C:\python\my_project01\build\tests\simple_test_suite from C:\python\my_project01\my_project\tests\simple_test_suite\clouds\test_cloud\test_cloud.s3cld
Use the following command to run tests manually:
"C:\Python311-32\python.exe" "py_tests_runner.py" "--err_to_stderr" "C:\python\my_project01\my_project\tests\simple_test_suite\src"
Press Enter to continue...

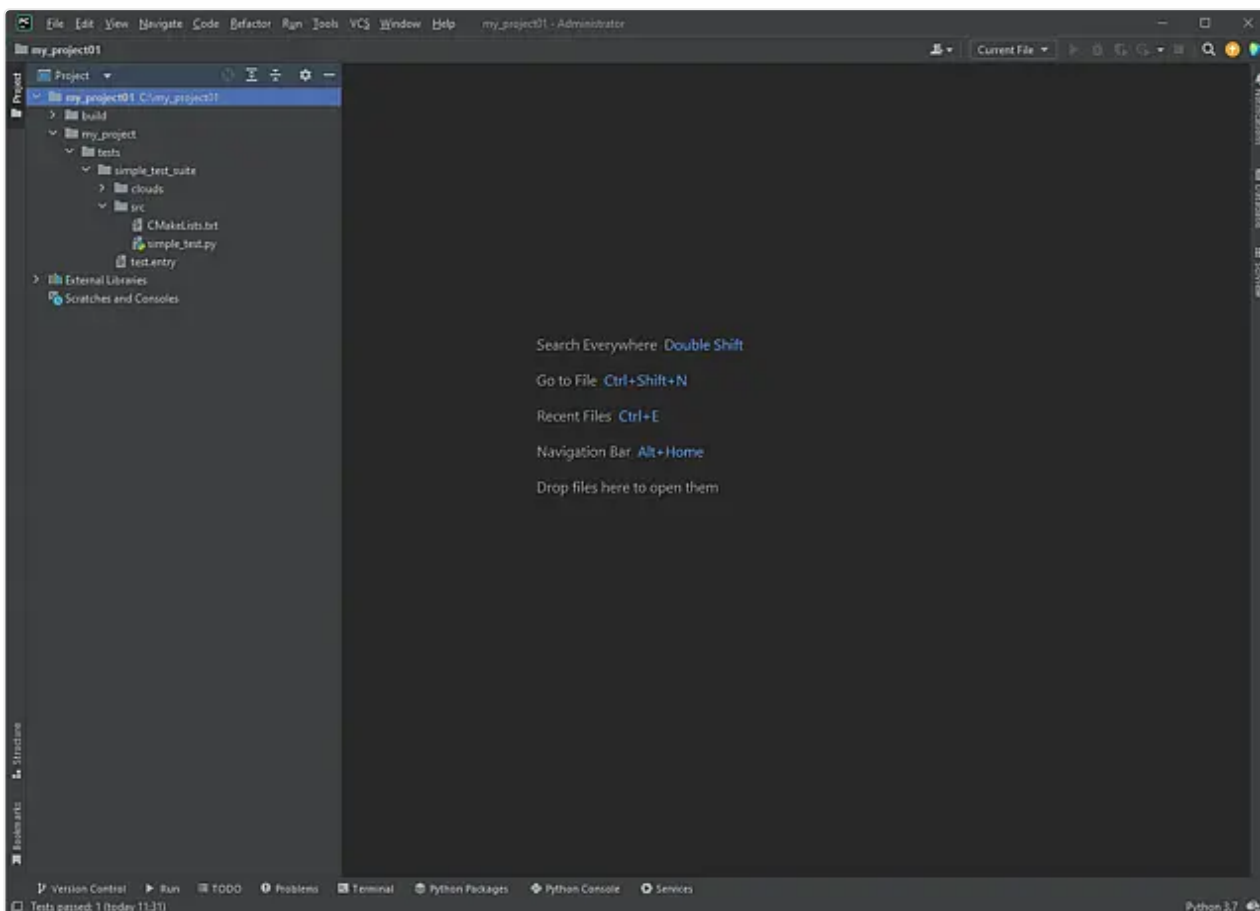
```

Запустите снова консоль, чтобы открылось новое окно. В нем перейдите в каталог `build\tests\simple_test_suite\run` и выполните команду:

Запуск тестов на Python в PyCharm

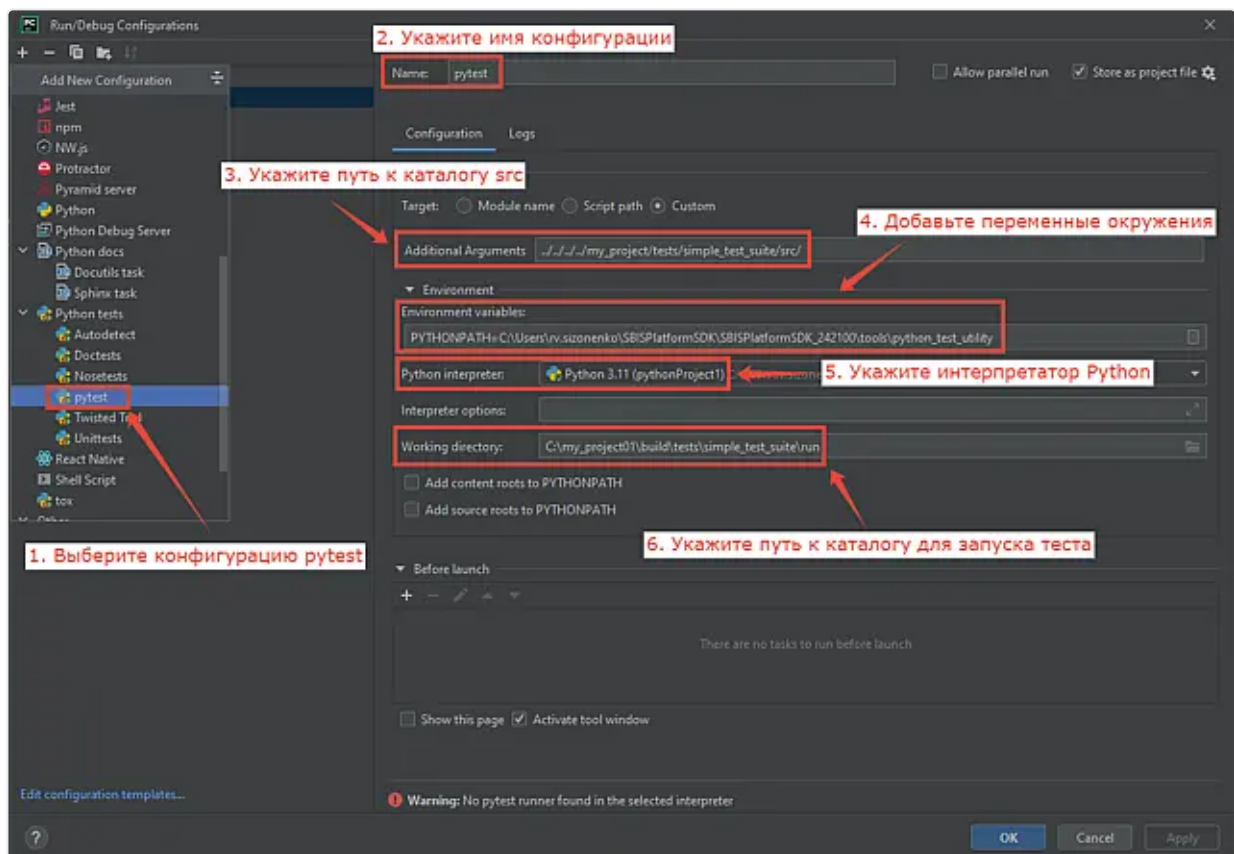
Для запуска тестов на [pytest](#) и [unittest](#) в PyCharm выполните следующие шаги:

1. Запустите тестовый фреймворк для развертывания рабочей директории теста. Подробно описано [здесь](#).
2. [Проверьте версию SBIS SDK](#) во всех файлах проекта, где она указана, иначе при запуске теста возникнет ошибка.
3. Откройте каталог с проектом в PyCharm:



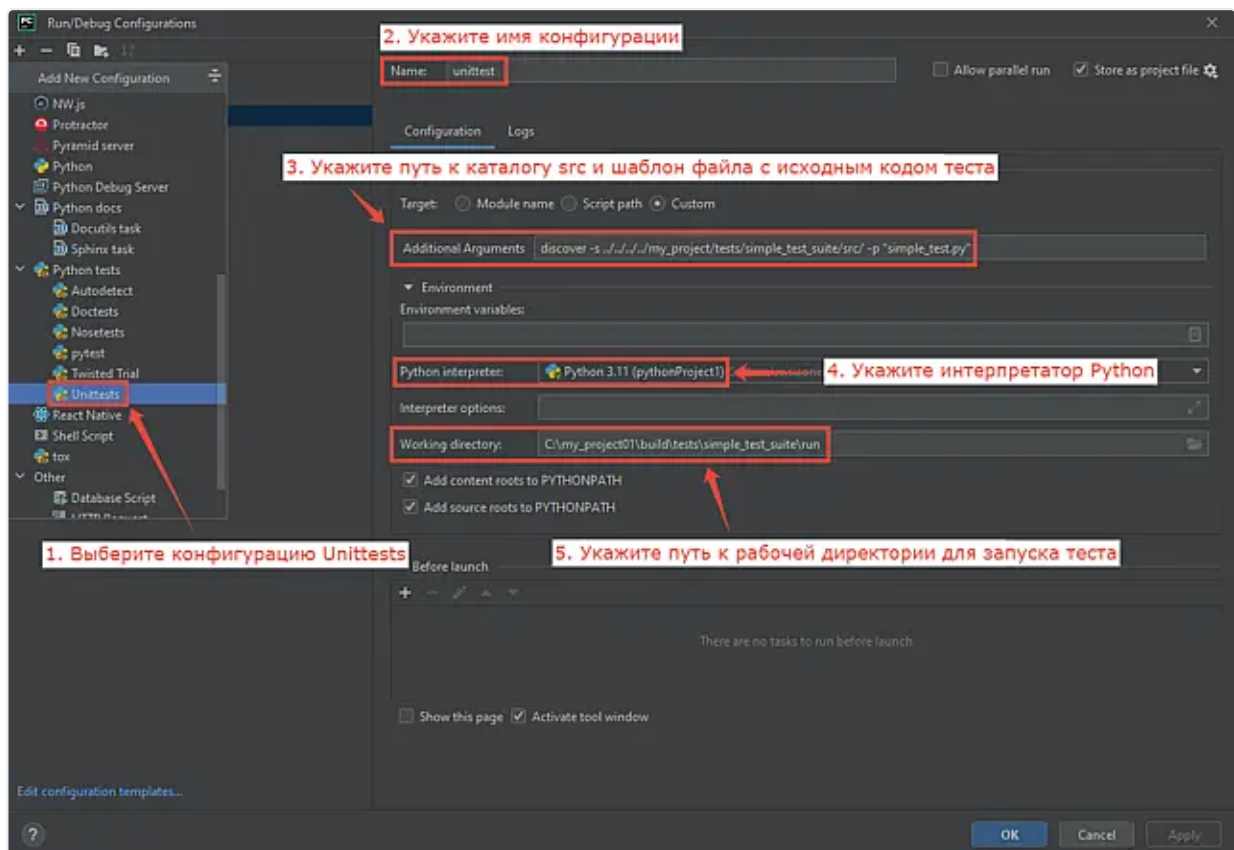
4. Перейдите в меню **Run** → **Edit Configurations**. Нажмите "+" и добавьте конфигурации:

- для `pytest` - выберите конфигурацию `pytest` и заполните поля, как на скриншоте ниже.



В поле **Additional Arguments** укажите путь до каталога **src**.

- для unittest - выберите конфигурацию Unittests и заполните поля, как на скриншоте ниже.



В поле **Additional Arguments** в аргументе `-s` укажите путь до каталога **src**, а в аргументе `-p` - шаблон имени файла с исходным кодом теста.

В поле **Environment variables** для Windows переменные окружения не указываются.

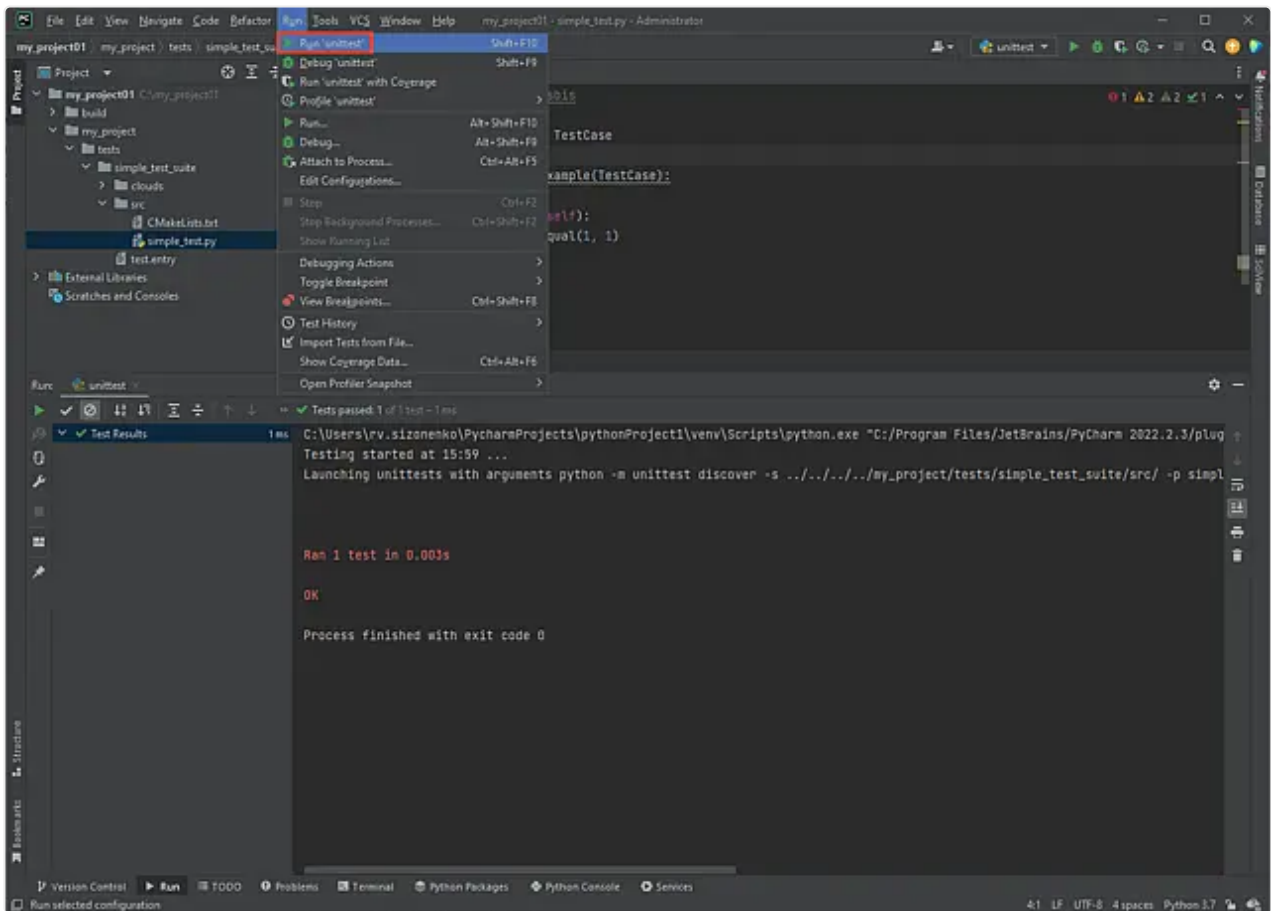
Важно!

При настройке на RedHat и macOS в поле **Environment variables** дополнительно укажите переменную:

- `LD_LIBRARY_PATH=.` - для RedHat

- DYLD_LIBRARY_PATH=. - для macOS

5. Для запуска теста перейдите в меню **Run** и запустите тест для соответствующей конфигурации, например, для unittest:



Запуск автотестов на Python в PyCharm

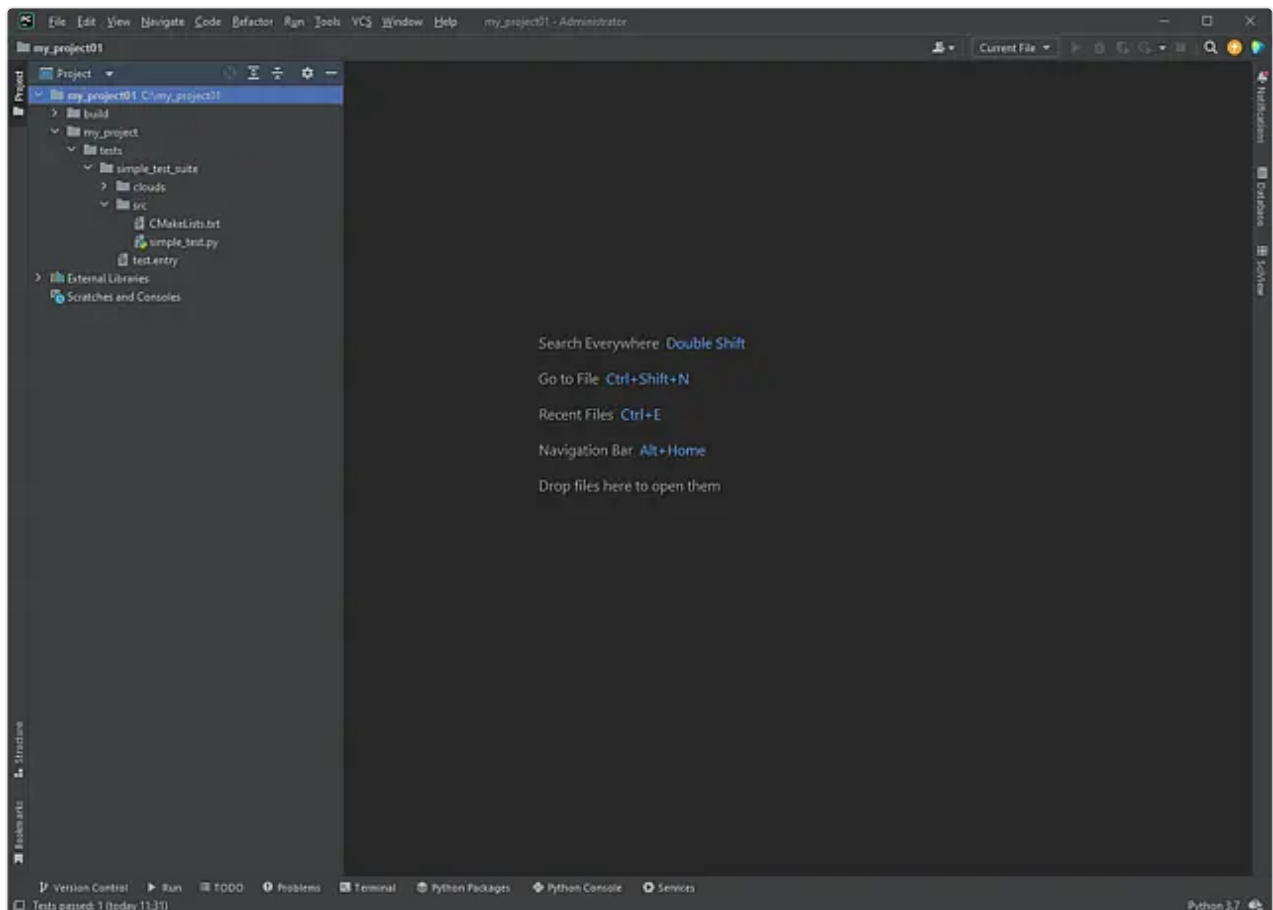


Важно!

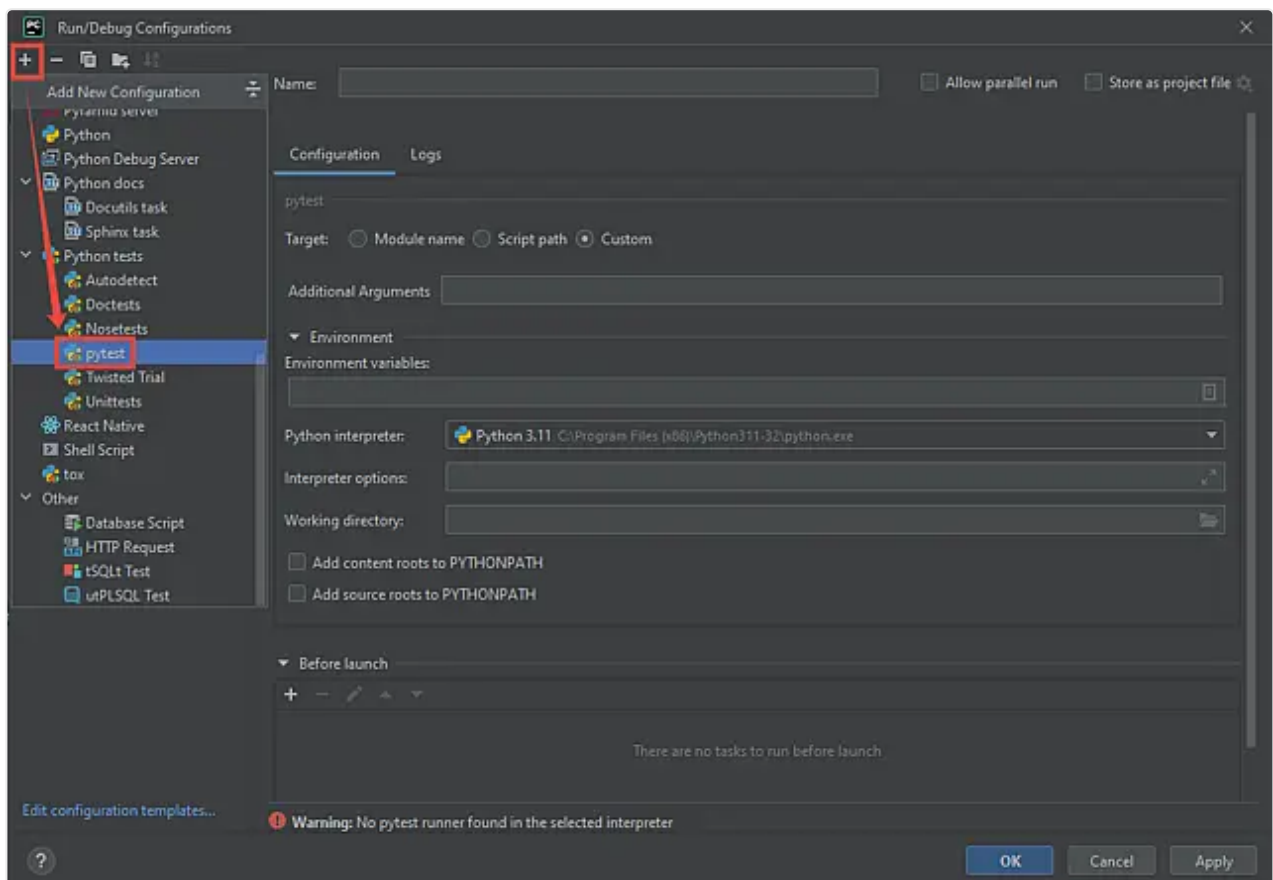
Настройка запуска автотестов в PyCharm выполняется для [pytest](#). При запуске с unittest возможны проблемы.

Для настройки запуска автотестов выполните следующие шаги:

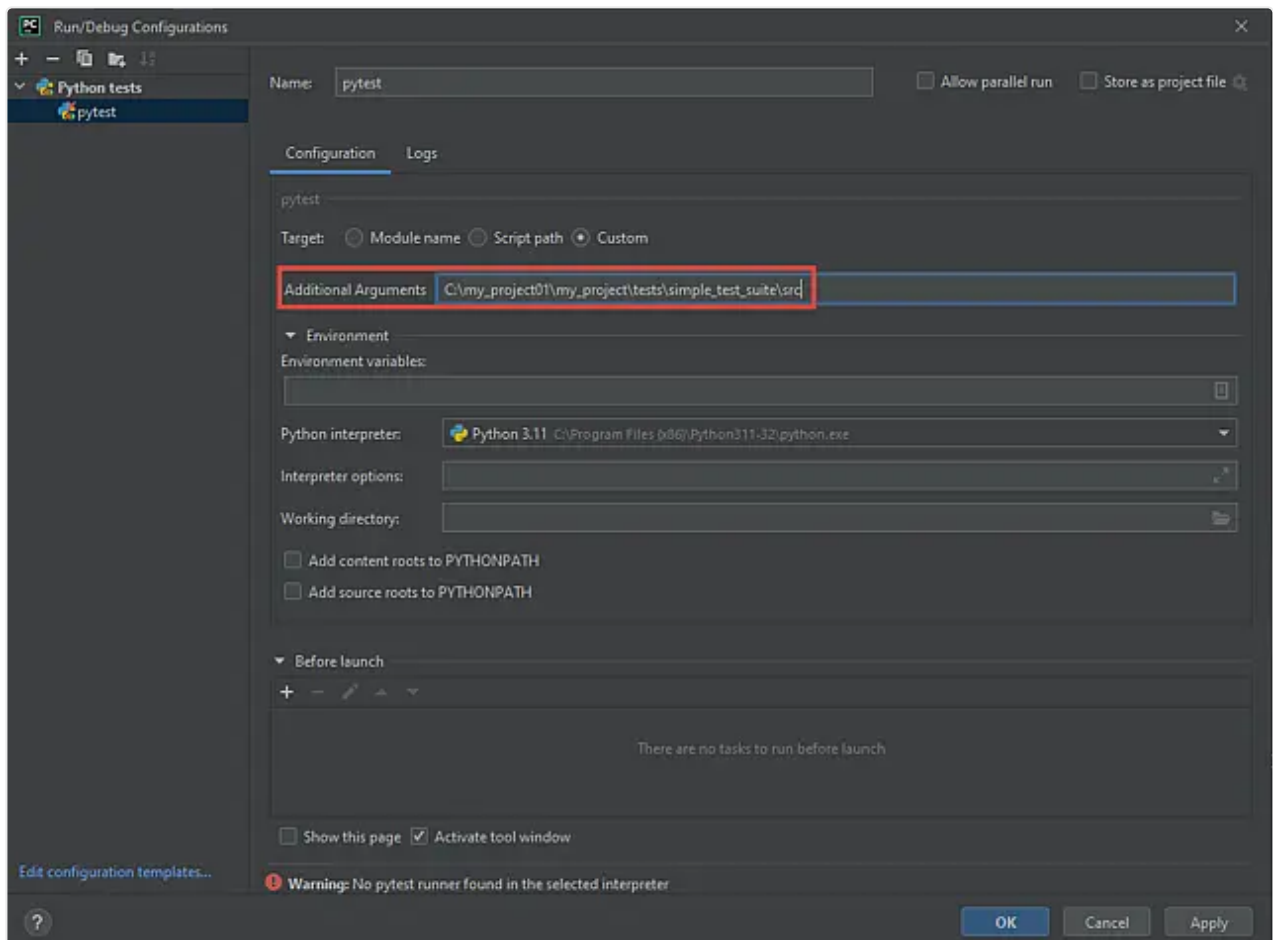
1. [Запустите тест](#).
2. Откройте каталог с проектом в PyCharm:



3. Перейдите в меню **Run** → **Edit Configurations**. Нажмите "+" и добавьте конфигурацию pytest:

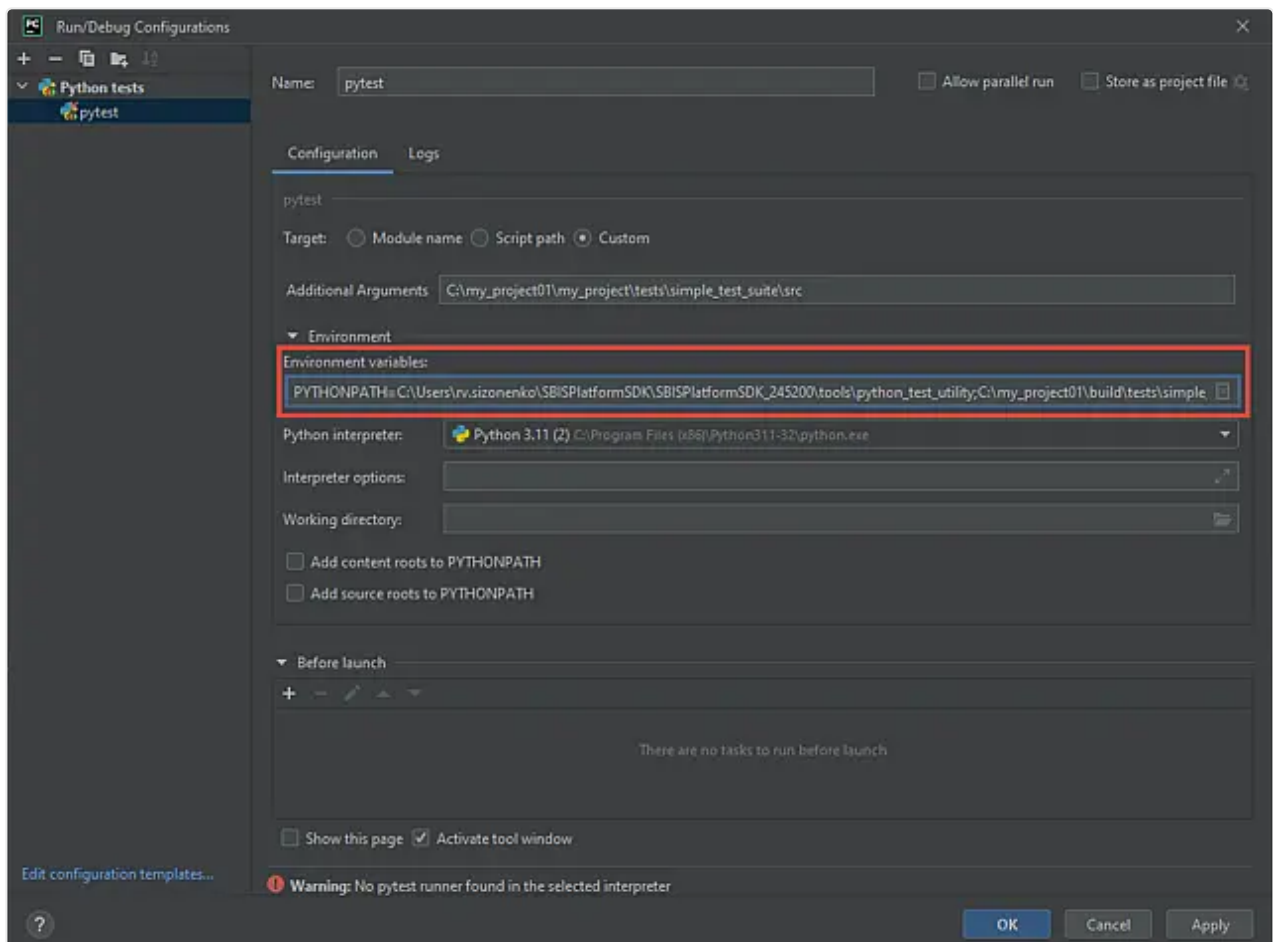


4. В поле **Additional Arguments** укажите путь до директории с тестами (**src**):



5. В поле **Environment variables** укажите переменные окружения:

PYTHONPATH=.../<путь к директории SDK>/tools/python_test_utility;.../<путь к директории run>



Пример для Windows

```
PYTHONPATH=C:\Users\rv.sizonenko\SBISPlatformSDK\SBISPlatformSDK_245200\tools\python_test_utility;C:\my_project01\build\tests\simple_test_suite\run
```

Для RedHat и macOS пути перечисляются через двоеточие!

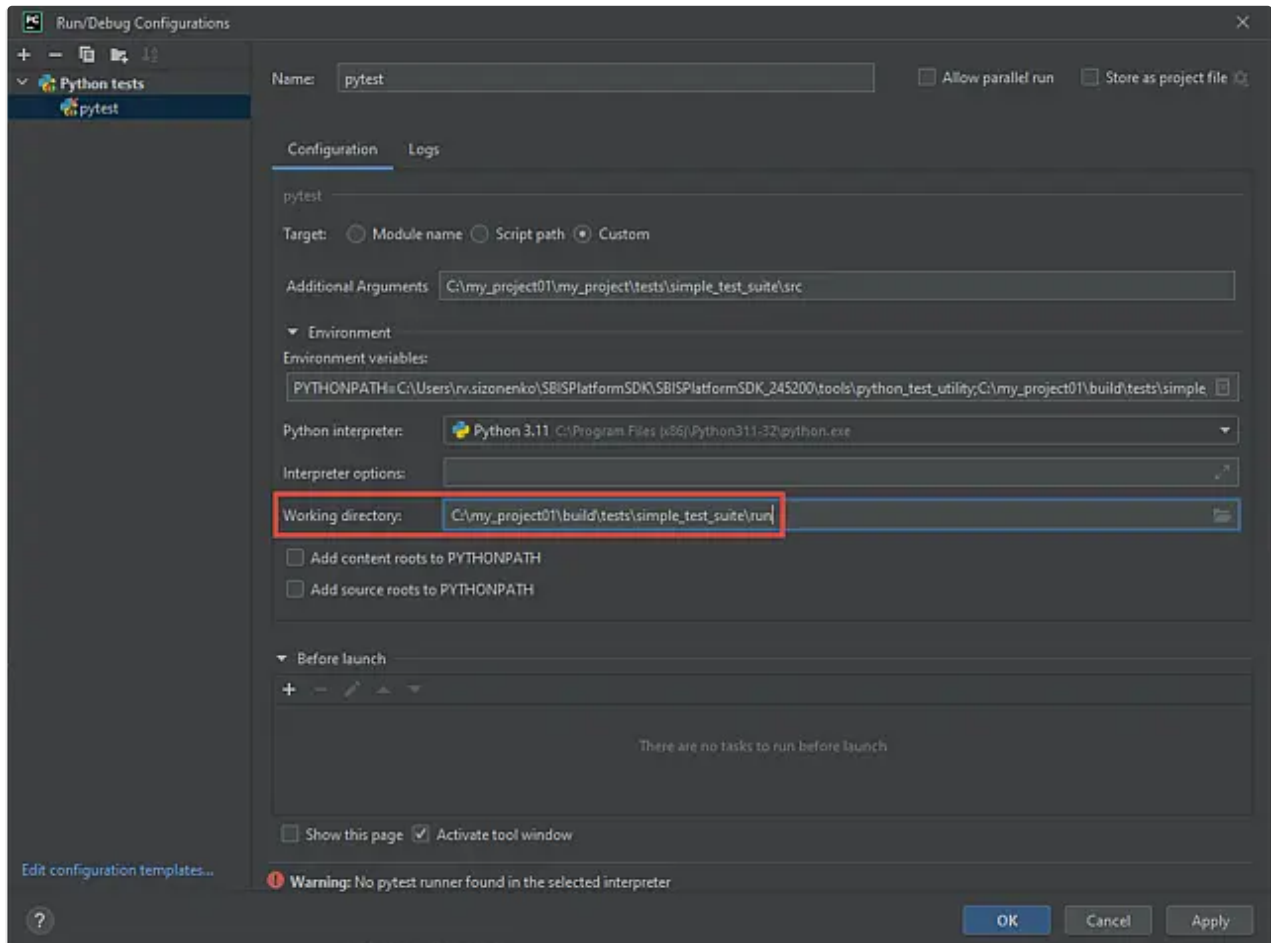


Важно!

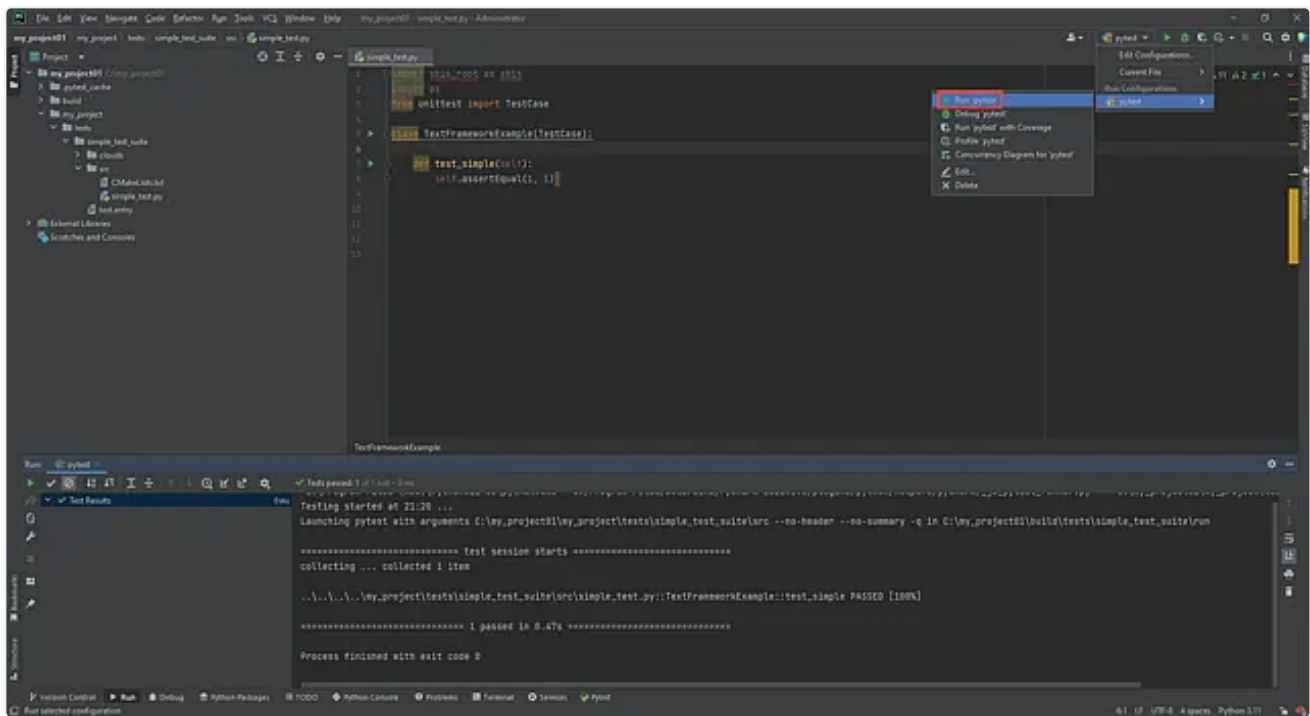
При настройке на RedHat и macOS в поле **Environment variables** дополнительно укажите переменную:

- LD_LIBRARY_PATH=. - для RedHat
- DYLD_LIBRARY_PATH=. - для macOS

6. В поле **Working directory** укажите путь до директории **run**, которая создается в директории **build** после запуска теста.



После применения настроек будет доступен запуск автотестов с помощью PyCharm в режиме обычного запуска, отладки или запуска с оценкой покрытия кода.



Пропуск запуска тестов

Для отключения запуска конкретного теста необходимо исключить его проект из сборки в файле **CMakeLists.txt**. Тогда CMake не сгенерирует файл <сборочная директория>/parameters/<имя теста>.txt и test_framework пропустит запуск этого теста с соответствующим сообщением:

```
Test "<имя теста>" skipped: project was not built, no such file
parameters/<имя теста>.txt
```

Пример для пропуска теста в сборках для macOS:

```
1 // нужно добавить в начало файла src/CmakeLists.txt строки
2
3 if( SBIS_OS STREQUAL "macos" ) // здесь может быть любое условие
  пропуска теста
4     return()
5 endif()
```

Анализ результатов выполнения теста

После выполнения теста получится следующая структура каталога:

```
├── build
│   ├── tests
│   │   ├── my_test_suite
│   │   │   ├── run
│   │   │   │   ├── tmp
│   │   │   │   ├── <каталог с дополнительным сервисом>
│   │   │   │   │   ├── logs
│   │   │   │   │   └── ...
│   │   │   │   ├── <каталог с дополнительным сервисом>
│   │   │   │   ├── logs
│   │   │   │   └── output.txt
│   └── my_project
```

- В файле **output.txt** находятся результаты выполнения ваших тестов.

- В каталоге **logs** находятся логи сервисов, которые работали во время выполнения теста.
- В каталоге **tmp** находятся логи разворота тестового окружения.

Режим разворота баз данных

Каркас модульного тестирования при подготовке окружения для теста мобильного контроллера, идентифицируемого опцией `mobile_controller`, использует особую логику разворота баз данных SQLite, входящих в состав сервисов контроллера, варьирующуюся в зависимости от режима запуска модульного теста.

- При автоматическом запуске теста базы данных SQLite разворачиваются в оперативной памяти ([In-Memory Databases](#)).
- В отладочном режиме теста (команда [debug](#)) базы данных разворачиваются в виде обычных файлов на диске.

Разворот баз данных в оперативной памяти позволяет реализовывать тестовые сценарии с большими накладными расходами доступа к БД, требовательные к производительности исполняющего узла (в особенности на загруженных сборочных машинах).

Данная логика подходит для большей части тестов мобильных микросервисов и контроллеров. Однако в некоторых случаях требуется работать с файлами баз данных, к примеру, при наличии заранее подготовленных тестовых данных. Также может потребоваться задать разворот в оперативной памяти и в отладочном режиме.

Для реализации подобных сценариев каркас тестирования предоставляет следующие варианты переопределения режима разворота баз данных:

- Задание режима разворота баз данных в описании теста с помощью булевой опции `in_memory_databases`:

```
1 {
2     "mobile_controller": "test-profile-controller",
3     "project_under_test": "../..apps/profiles service.s3srv",
4     "in_memory_databases": false
5 }
```

в файле `test.entry` отключает разворот в памяти баз данных всех тестовых проектов.

- Задание режима разворота в файле конфигурации тестового сервиса:

```
1 [Базовая конфигурация]
2 __sbis__in_memory_dbs = Нет
```

В файле `sbis-rpc-service.ini` включается режим разворота баз данных в файловой системе для конкретного тестового проекта.

Режим игнорирования ошибок загрузки сервиса

Раньше для тестирования прикладного кода в состав сервиса по [зависимостям из s3mod](#) требовалось добавить множество ненужных модулей. Это приводило к созданию пустых `mock`-модулей, которые размещались рядом с тестами.

Для решения этой проблемы в файл `test.entry` добавляется параметр `"skip_missing": true`. Его необходимо указывать для самого теста, а также для каждого дополнительно разворачиваемого сервиса.

"skip_missing" активирует параметр конфигурации:

```
[ "Базовая конфигурация" ]
"ИгнорироватьОшибкиЗагрузкиМодулей" = Да
```

который, в свою очередь, частично отключает проверки целостности зависимостей сервисов.

Python API

Для встраивания тестового фреймворка в другие скрипты на Python необходимо добавить путь к пакету в переменную `PYTHONPATH` перед запуском интерпретатора или в список `sys.path` перед импортом пакета.

Пакет предоставляет единственную функцию `run`. Единственным обязательным параметром для вызова функции является путь к каталогу с тестовыми проектами, поэтому запуск тестов сводится к выполнению следующих команд:

```
1 import test_framework
```


2

3 `test_framework.run( <путь к каталогу с тестовыми проектами> )`

Таблица 2. Необязательные параметры функции run.

Имя	Тип	Значение по умолчанию	Описание
tests_only	list	list()	Список имен тестов, которые необходимо выполнить. Если список пуст, то выполняются все найденные тесты.
jinnee_dirs	list	list()	Список каталогов, в которых необходимо искать jinnee-utility . Если список пуст, то каталог определяется переменной окружения Jinnee .
jobs	int	1	Количество параллельно выполняемых тестов.
debug	bool	False	Включить режим отладки, при котором для каждого теста будет создано только тестовое окружение без запуска самого теста. Имеет смысл только при интерактивной работе с консолью.
run_cleaner	bool	False	Выполнить поиск и удаление "мусорных" БД, т. е. БД оставшихся в результате аварийных прерываний работы тестовой оболочки. Параметр рекомендуется всегда устанавливать в True при выполнении тестов на серверах сборки.
verbose	bool	False	Выводить подробную информацию о ходе выполнения тестов. Для упрощения выявления и решения проблем параметр рекомендуется всегда задавать True при выполнении тестов на серверах сборки.
pghost	str	"localhost"	Имя сервера БД.
pgport	str	"5432"	Порт сервера БД.
pguser	str	"postgres"	Имя пользователя на сервере БД.
pgpass	str	"<пароль вашей локальной БД>"	Пароль пользователя.

Функция возвращает кортеж вида (количество тестов, количество ошибок).

Как подключить тесты к merge request?

1. Заливаем тесты в наш репозиторий.

2. Пишем поручение [Сергею Сахаревичу](#) с формулировкой:

- *Прошу собрать БЛ-тесты для нашего репозитория, запускаемые при пуше:*
 - *Адрес проверяемого репозитория: ...*
 - *Путь до директории с тестами: ...*
 - *Путь до CMakeLists: ...*

3. Пишем поручение ответственному за сборку вашего приложения с формулировкой:

Прошу включать автозаворот добросок при фейле тестов по веткам